SOC L1 LAB · INC-002

INC-002 Detection Rules

Seven rules, six execution paths, and the gaps they leave

INCIDENT	INC-002 — Privilege-assignment detection rules
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Design principle
2. Rule inventory
3. The rules
4. Rule-by-rule
5. Test matrix
6. Coverage and gaps
7. Tuning
8. Deployment
9. MITRE ATT&CK
10. Caveats

These are the Wazuh rules used for INC-002, plus the INC-001 persistence rule they depend on for context. Every rule below was loaded in Wazuh 4.14.7 and exercised against a live Debian 12 endpoint on 2026-10-31.

The file is `rules/local_rules.xml`. The same content is reproduced here so the detection logic can be reviewed without the repository.

---

1. Design principle

Stock Wazuh tells you **that** a user exists (5902, level 8). It does not tell you **that the user was given root capability**. The rules below close that gap by naming the privilege assignment:

- anchor on the audit command that performs the change;
- match the binary and the group argument, not a file diff;
- raise the level to 12 so the alert is not queued behind inventory noise;
- put the affected user and group in the description.

They are intentionally local rules in the `100100` – `100199` range.

---

2. Rule inventory

ID	Trigger	Level	MITRE
100120	FIM: `authorized_keys` added or modified	12	T1098.004, T1078
100121	`usermod` with group argument `sudo` / `wheel` / `admin`	12	T1098, T1548.003
100122	SSH login by an account named `svc-*`	10	T1078
100131	Service-account login after the 40112 brute-force correlation	12	T1078, T1110
100123	`gpasswd -a` to `sudo` / `wheel` / `admin`	12	T1098, T1548.003
100124	Debian `adduser <user> <group>` to a privileged group	12	T1098, T1548.003
100125	`useradd -G <privileged-group> <user>`	12	T1136.001, T1098
100126	`usermod -a -G <privileged-group> <user>`	12	T1098, T1548.003

---

3. The rules

```
<group name="local,syscheck,">

  <rule id="100120" level="12">
    <if_sid>550,554</if_sid>
    <match>authorized_keys</match>
    <description>FIM: SSH authorized_keys added or modified (possible persistence)</description>
```

```
    <mitre>
      <id>T1098.004</id>
      <id>T1078</id>
    </mitre>
    <group>syscheck,persistence,</group>
  </rule>

</group>

<group name="local,audit,">

  <rule id="100121" level="12">
    <if_sid>80792</if_sid>
    <field name="audit.exe">/usr/sbin/usermod</field>
    <field name="audit.execve.a2" type="pcre2">^(sudo|wheel|admin)$</field>
    <description>Privilege assignment: user $(audit.execve.a3) added to privileged group
$(audit.execve.a2) via usermod</description>
    <mitre>
      <id>T1098</id>
      <id>T1548.003</id>
    </mitre>
    <group>audit_command,privilege_escalation,</group>
  </rule>

  <rule id="100126" level="12">
    <if_sid>80792</if_sid>
    <field name="audit.exe">/usr/sbin/usermod</field>
    <field name="audit.execve.a1">-a</field>
    <field name="audit.execve.a2">-G</field>
    <field name="audit.execve.a3" type="pcre2">^(sudo|wheel|admin)$</field>
    <description>Privilege assignment: user $(audit.execve.a4) added to privileged group
$(audit.execve.a3) via usermod -a -G</description>
    <mitre>
      <id>T1098</id>
      <id>T1548.003</id>
    </mitre>
    <group>audit_command,privilege_escalation,</group>
  </rule>

  <rule id="100123" level="12">
    <if_sid>80792</if_sid>
    <field name="audit.exe">/usr/bin/gpasswd</field>
    <field name="audit.execve.a1">-a</field>
    <field name="audit.execve.a3" type="pcre2">^(sudo|wheel|admin)$</field>
    <description>Privilege assignment: user $(audit.execve.a2) added to privileged group
$(audit.execve.a3) via gpasswd</description>
    <mitre>
      <id>T1098</id>
      <id>T1548.003</id>
    </mitre>
    <group>audit_command,privilege_escalation,</group>
```

```
  </rule>

  <rule id="100124" level="12">
    <if_sid>80792</if_sid>
    <field name="audit.exe">/usr/bin/perl</field>
    <field name="audit.execve.a1">/usr/sbin/adduser</field>
    <field name="audit.execve.a3" type="pcre2">^(sudo|wheel|admin)$</field>
    <description>Privilege assignment: user $(audit.execve.a2) added to privileged group
$(audit.execve.a3) via adduser</description>
    <mitre>
      <id>T1098</id>
      <id>T1548.003</id>
    </mitre>
    <group>audit_command,privilege_escalation,</group>
  </rule>

  <rule id="100125" level="12">
    <if_sid>80792</if_sid>
    <field name="audit.exe">/usr/sbin/useradd</field>
    <field name="audit.execve.a1">-G</field>
    <field name="audit.execve.a2" type="pcre2">^(sudo|wheel|admin)$</field>
    <description>Privilege assignment: new account $(audit.execve.a3) created in privileged group
$(audit.execve.a2)</description>
    <mitre>
      <id>T1136.001</id>
      <id>T1098</id>
    </mitre>
    <group>audit_command,privilege_escalation,</group>
  </rule>

</group>

<group name="local,syslog,sshd,">

  <rule id="100122" level="10">
    <if_sid>5715</if_sid>
    <match>for svc-</match>
    <description>SSH: interactive login by a service account</description>
    <mitre>
      <id>T1078</id>
    </mitre>
    <group>authentication_success,</group>
  </rule>

  <rule id="100131" level="12">
    <if_sid>40112</if_sid>
    <match>for svc-</match>
    <description>SSH: service-account login after brute-force correlation</description>
    <mitre>
      <id>T1078</id>
      <id>T1110</id>
```

```
    </mitre>
    <group>authentication_success,brute_force,</group>
  </rule>

</group>
```

---

4. Rule-by-rule

100121 — `usermod -aG` / `-G`

- **Matches:** `/usr/sbin/usermod`, with the group in `a2`.
- **Covers:** `usermod -aG sudo alice` and `usermod -G sudo alice`.
- **Tested:** 05:01:28.682, `usermod -aG sudo svc-test`.
- **False positives:** legitimate onboarding and configuration management.
- **Limit:** options before the group shift the index; the `-a -G` form is handled by 100126.

100126 — `usermod -a -G`

- **Matches:** `a1=-a`, `a2=-G`, `a3=<privileged group>`.
- **Covers:** `usermod -a -G sudo alice`, which is what Debian `adduser` calls internally.
- **Tested:** 05:03:40.524 (manual) and 05:03:46.527 (inside `adduser`).
- **False positives:** same as 100121.
- **Limit:** a duplicate alert is expected when `adduser` is used: 100124 catches the wrapper and 100126 catches the internal `usermod`. Keep both, or suppress one with `ignore` if the duplication is noisy.

100123 — `gpasswd -a`

- **Matches:** `/usr/bin/gpasswd`, `a1=-a`, `a3=<privileged group>`.
- **Covers:** `gpasswd -a alice sudo`.
- **Tested:** 05:03:36.524.
- **Important:** the `a1=-a` condition excludes `gpasswd -d`, which removes a user and would otherwise produce a false privilege-assignment alert. This was caught during testing.
- **Limit:** `gpasswd -M` (set the member list) is not covered.

100124 — Debian `adduser`

- **Matches:** `/usr/bin/perl`, `a1=/usr/sbin/adduser`, `a3=<privileged group>`.
- **Covers:** `adduser alice sudo`.
- **Tested:** 05:03:46.525, execve vector `perl /usr/sbin/adduser svc-test2 sudo`.
- **Why `perl`:** Debian's `adduser` is a Perl script, so the audit `exe` is `/usr/bin/perl`, not `/usr/sbin/adduser`. A rule matching the script path alone will never fire.
- **Limit:** another Perl wrapper invoking `adduser` would match; require the exact `a1` path to keep it narrow.

100125 — `useradd -G`

- **Matches:** `/usr/sbin/useradd`, `a1=-G`, `a2=<privileged group>`.

- **Covers:** `useradd -G sudo alice`, the canonical ordering.
- **Tested:** 05:03:50.525.
- **Limit:** `useradd -m -s /bin/bash -G sudo alice` shifts the group to `a5`. The rule intentionally covers the short form; the FIM + group-baseline control covers the rest. Add a second rule for the longer ordering if that is the standard in your estate.

100122 — service-account SSH login

- **Matches:** rule 5715 (SSH authentication success) whose log contains `for svc-`.
- **Tested:** 05:03:54.736, `Accepted password for svc-test`.
- **Why not `<field name="dstuser">`:** Wazuh rejects `<field>` for `dstuser` with "Field 'dstuser' is static." Static decoder fields are matched with `<match>` or `<regex>` instead.
- **Covers:** the `svc-*` naming convention. An account called `backup` or `deploy` needs the pattern extended.
- **False positives:** a service that legitimately uses SSH. Allow-list it explicitly rather than removing the rule.

100131 — precedence fix for 100122

- **Problem:** stock 40112 matches the successful login after a brute-force burst and wins rule evaluation, so the sibling 100122 never fires. The service-account login is hidden behind the brute-force alert.
- **Fix:** 100131 is a child of 40112 with the same `for svc-` match. It fires when the success follows failures, and names the account.
- **Tested:** 8 failures + `Accepted password for svc-test` → **100131**; a normal `svc-test` login → **100122**.
- **Limitation:** two rules express one intent. This is preferable to suppressing 40112 or accepting a blind spot.

100120 — persistence context

- **Matches:** FIM 550/554 with `authorized_keys` in the log.
- **Tested:** INC-001, 2026-10-30 15:22:22, path and SHA-256 in the alert.
- **Why here:** the strongest combination for INC-002 is a new privileged account *plus* a key file. 100120 supplies the second half.

---

5. Test matrix

Run these on a disposable account. Each line is a separate action; allow two seconds between them so the alerts are easy to attribute.

Action	Expected
`useradd -m -s /bin/bash svc-test`	5902, 5901, FIM 554
`usermod -aG sudo svc-test`	**100121**
`usermod -a -G sudo svc-test`	**100126**
`gpasswd -a svc-test sudo`	**100123**
`gpasswd -d svc-test sudo`	none from 100123 (by design)
`adduser svc-test sudo`	**100124** + **100126**
`useradd -G sudo svc-test2`	**100125**
SSH login as `svc-test`	**100122**
Write `~/.ssh/authorized_keys`	**100120**

Example:

```
sudo useradd -m -s /bin/bash svc-test
echo 'svc-test:Summer2026!' | sudo chpasswd

sudo usermod -aG sudo svc-test        # 100121
sudo gpasswd -d svc-test sudo
sudo gpasswd -a svc-test sudo         # 100123
sudo gpasswd -d svc-test sudo
sudo usermod -a -G sudo svc-test      # 100126
sudo gpasswd -d svc-test sudo
sudo adduser svc-test sudo            # 100124 + 100126
sudo gpasswd -d svc-test sudo
sudo useradd -G sudo svc-test2        # 100125

ssh svc-test@127.0.0.1                # 100122

sudo gpasswd -d svc-test sudo
sudo userdel -r svc-test
sudo userdel -r svc-test2
```

Expected counts in the test window: one each of 100121, 100123, 100124, 100125, 100122; two of 100126 (manual plus the `adduser` internal call); 100120 when the key is written.

5.1 False-positive validation

The first version used an unanchored alternation, `sudo|wheel|admin`. Wazuh treats a `<field>` value as a regular expression, so that pattern matched any value containing the substring. Legitimate groups named `sudo-audit`, `admin-tools`, and `wheelhouse` produced privilege-assignment alerts.

Test group	Unanchored `sudo|wheel|admin`	Anchored PCRE2
`appusers` (non-privileged)	silent	silent
`sudo-audit`	**100121 fired**	silent
`admin-tools`	**100121, 100124, 100126 fired**	silent
`wheelhouse`	**100121 fired**	silent
`sudo` (exact)	100121 fired	100121 fired

The same trap affected each path: `gpasswd -a testuser sudo-audit` fired 100123, `adduser testuser admin-tools` fired 100124 and 100126, and `useradd -G sudo-audit testuser5` fired 100125.

The fix is an exact match:

```
<field name="audit.execve.a3" type="pcre2">^(sudo|wheel|admin)$</field>
```

Wazuh rejects `^...$` without `type="pcre2"` (syntax error on the tag), which is why the first version omitted the anchors. After the change, the substring groups were silent and the exact `sudo` operation still fired.

This is the value of running legitimate activity through the rules: a rule that never false-positives in a demo can still be wrong in a real estate with `sudo-audit` or `admin-tools`.

5.2 100120 lookalike test

Tested on 2026-10-31 with legitimate activity on the endpoint:

Action	Result
`ssh-keygen`, `.bashrc` edit	silent
`/home/analyst/notes/authorized_keys.txt`	**100120 fired** (false positive)
`/home/analyst/.ssh/authorized_keys.bak`	**100120 fired** (false positive)
Real `~/.ssh/authorized_keys` rotation	fired (expected)
Restore of the original file	fired (expected)

The rule matches the substring `authorized_keys`, so a lookalike filename triggers it. Exact-path attempts all failed:

- `<match>authorized_keys'</match>` — the match engine normalizes punctuation, so the quote is never seen.
- `<regex>authorized_keys'</regex>`, `<regex>authorized_keys[^.]</regex>`, and `<regex>authorized_keys.modified</regex>` — no match against the FIM log.
- `<field name="syscheck.path" type="pcre2">` — accepted by the parser but never evaluated; the FIM path is not a rule field.

Keeping the broad match is the honest choice: it catches the real file. The cost is lookalike names and legitimate key rotation. Mitigations:

- do not name backups `authorized_keys.*` inside FIM-monitored paths;
- keep a key-fingerprint baseline and allow-list expected keys;
- match legitimate rotations to a change ticket.

A custom decoder that extracts the path into a dynamic field is the next step if the lookalike noise becomes material.

---

6. Coverage and gaps

Path	Covered by	Notes
`usermod -aG`	100121	Primary Debian path
`usermod -G`	100121	Same index
`usermod -a -G`	100126	`adduser` internal path
`gpasswd -a`	100123	Additions only
`adduser`	100124 (+100126)	Perl wrapper
`useradd -G`	100125	Short form only
Direct `/etc/group` edit	**not covered**	FIM 550 + membership baseline
`vigr` or editor	**not covered**	FIM 550 + baseline
`gpasswd -M`	**not covered**	Add a rule for `a1=-M` if used
Long options (`--groups`)	**not covered**	Argument index changes
LDAP/SSSD group changes	**not covered**	Different telemetry
Custom privileged group	update alternation/list	`sudo|wheel|admin` here

The baseline is the backstop:

```
getent group sudo  > /var/lib/soc/sudo-group.baseline
getent group wheel >> /var/lib/soc/sudo-group.baseline 2>/dev/null || true
getent group admin >> /var/lib/soc/sudo-group.baseline 2>/dev/null || true
# periodic
getent group sudo | diff -u /var/lib/soc/sudo-group.baseline -
```

---

7. Tuning

7.1 Privileged groups

The rules use anchored PCRE2, `^(sudo|wheel|admin)$`. Keep the anchors: an unanchored alternation matches `sudo-audit`, `admin-tools`, and `wheelhouse`. For a different group, either:

- change the alternation in 100121/100123/100124/100125/100126, or
- move the list into a CDB lookup, as the stock audit rules do with `etc/lists/audit-keys`.

A CDB list is better when the set changes often. Test the lookup with `wazuh-logtest` before deploying.

7.2 Argument order

Audit exposes one field per argument. The same logical command can put the group at a different index:

```
usermod -aG sudo alice          a2=sudo
usermod -a -G sudo alice        a3=sudo
usermod -G sudo alice           a2=sudo
useradd -G sudo alice           a2=sudo
useradd -m -s /bin/bash -G sudo alice   a5=sudo
```

The rules cover the first four. The last is covered by FIM and the baseline. If your provisioning standard uses the long form, add a rule for the specific index rather than loosening all of them.

7.3 Duplicate alerts

`adduser` produces 100124 and 100126 because it wraps `usermod`. That is acceptable defence in depth. If it is noisy, add `ignore="60"` to 100126 or make 100124 a parent and 100126 a child, and verify the suppression does not swallow a real manual `usermod -a -G`.

7.4 FIM and baseline

Do not try to make these rules cover every possible write to `/etc/group`. Use:

- FIM 550 on `/etc/group` with `report_changes="yes"`;
- a nightly membership baseline;
- a drift alert on any difference.

That is the only path that catches `vigr`, an editor, or a direct rewrite.

---

8. Deployment

The manager mounts the repository's `rules/` directory:

```yaml
volumes:
  - /home/luqman/Documents/Eterna/soc-l1-lab/rules:/var/ossec/etc/rules:ro
```

```
cd "$HOME/wazuh-docker/single-node"
docker compose up -d wazuh.manager
sleep 20

for id in 100120 100121 100122 100123 100124 100125 100126; do
```

```
  printf '%s: ' "$id"
  docker exec single-node-wazuh.manager-1 \
    grep -c "id=\"$id\"" /var/ossec/etc/rules/local_rules.xml
done
```

Expected: `1` for each.

Validate the SSH rule without an account:

```
echo "Oct 31 05:03:54 soc-victim sshd[9001]: Accepted password for svc-test from 10.0.2.2 port 40000 ssh2" \
 | docker exec -i single-node-wazuh.manager-1 /var/ossec/bin/wazuh-logtest 2>&1 \
 | grep "100122"
```

Validate the audit rules against a disposable account using the matrix in §5, then delete it.

---

9. MITRE ATT&CK

Technique	ID	Rule
Create local account	T1136.001	100125, 5902
Account manipulation	T1098	100121, 100123, 100124, 100125, 100126
Abuse elevation control: sudo	T1548.003	100121, 100123, 100124, 100125, 100126
Valid accounts	T1078	100122
SSH authorized keys	T1098.004	100120

---

10. Caveats

- Tested on Debian 12 with auditd 3.0.9 and Wazuh 4.14.7. Binaries live in `/usr/sbin` and `/usr/bin`; a distribution with different paths needs the `audit.exe` values adjusted.
- The rules depend on `audit-wazuh-c` execve rules being loaded. Without auditd there is no evidence to match.
- The source address in this lab is the QEMU gateway for every session, so these rules deliberately key on the command and the group, not the source.
- A rule that fires is not an incident. Every hit still needs the change-control check described in `writeups/INC-002/03-detection-mitigation-triage.md`.